

Ingeniería y Calidad de Software

Jake and Drosh

24-06-2025

Pressman: Pruebas de aplicaciones convencionales - Capítulo 18

A continuación veremos un resumen explicando los **aspectos fundamentales** del Capítulo 18 sobre el **diseño de pruebas en aplicaciones convencionales**, organizados de manera clara y simplificada:

Enfoque general del capítulo

El capítulo aborda cómo **diseñar casos de prueba efectivos** para encontrar errores, sin que las pruebas se vean como algo destructivo, sino como una herramienta para mejorar la calidad del software.

Tipos principales de pruebas

1. Prueba de Caja Blanca (“glass-box testing”)

Se basa en el **conocimiento del código interno** del programa. Se analizan caminos lógicos y estructuras de control.

Técnicas clave:

- **Ruta básica (McCabe):** Asegura que cada línea del programa se ejecute al menos una vez.
- **Complejidad ciclomática:** Métrica que mide la cantidad de caminos lógicos. Más complejidad = más probabilidad de errores.
- **Prueba de bucles:** Se prueba el comportamiento de bucles en diferentes condiciones (una vez, varias veces, ninguna vez).

- **Prueba de condiciones:** Evalúa expresiones lógicas y decisiones del programa.
-

2. Prueba de Caja Negra

No considera el código interno. Solo importa **lo que entra y lo que sale** del sistema.

Técnicas clave:

- **Partición de equivalencia:** Agrupa entradas similares en clases para evitar probar todas.
 - **Análisis de valor límite:** Se enfoca en probar los “bordes” donde suelen aparecer errores (como mínimo y máximo).
 - **Pruebas basadas en gráficos:** Usan diagramas para representar y analizar flujos de datos, eventos o transacciones.
 - **Prueba ortogonal:** Usa combinaciones eficientes de entradas para probar distintas interacciones con pocos casos.
 - **Prueba basada en modelo:** Se derivan casos a partir de diagramas de estado o modelos de comportamiento.
-

Aplicaciones especializadas

El capítulo también explica cómo probar aplicaciones más complejas:

- **Interfaces gráficas (GUI):** Se evalúan tanto la funcionalidad como la interacción visual.
- **Arquitectura cliente-servidor:** Se crea un perfil del usuario y sus operaciones típicas.
- **Sistemas de tiempo real:** Requieren pruebas específicas por la sensibilidad al tiempo.
- **Documentación y centros de ayuda:** También deben probarse porque los errores allí afectan la experiencia del usuario.

Patrones de prueba

Así como en diseño existen patrones, también hay **patrones para diseñar pruebas**. Un ejemplo es **PairTesting**, donde dos personas trabajan juntas para idear y ejecutar casos de prueba, como en la programación en pareja.

Conclusión

El diseño de pruebas no se trata solo de verificar que todo funcione, sino de **pensar estratégicamente para encontrar errores antes de que causen daño**. Este capítulo enseña cómo hacerlo aplicando diversas técnicas según el tipo de aplicación y el contexto.
